

Universal Deployment Framework (UDF2)

Architecture Reference: v3.1 (Public Edition)

System	UDF2, two-layer Azure landing-zone planning and deployment framework
Version	3.1 (public edition)
Document revision	v3.1-public, 17 August 2026
Status	v3.1 — Public Edition
Date	17 August 2026
Audience	Cloud architects, platform engineers, technical reviewers
Context and Role	Sole designer and implementer. Built and hardened 2025–2026 as the author's own delivery tooling for repeatable, audit-ready deployments of regulated Azure estates; used in production consulting delivery. All numbers measured from the framework repository.

A note on names and numbers. This document describes a production system used to deploy and manage real Azure hub-and-spoke landing zones in regulated environments. All environment, company, and subscription identifiers are placeholders (env-alpha, env-bravo, {prefix}). All JSON samples are synthesized. Quantitative claims were re-verified against the repository on 2026-08-13; deployment durations are documented estimates from real deployments and are marked as such.

1. Executive Summary

1.1 Problem

An Azure hub-and-spoke landing zone is dozens of interdependent resources (VNETs, firewalls, VPN gateways, NAT gateways, route tables, NSGs, DNS zones, and peerings) that must be deployed in precise order, with configuration that differs by architecture profile. Manual portal deployment is error-prone, unreproducible, and impossible to audit. Moving infrastructure between subscriptions means rewriting every resource ID. Supporting both a firewall-centric and an NSG-centric architecture doubles the complexity.

A second, upstream problem compounds it: translating raw customer intent (an existing Azure estate, a CAF/ALZ artifact, or a text design sheet) into a validated, deployable inventory is itself an unstructured manual process with no systematic bridge to the deployment tooling.

1.2 Solution

UDF2 is a two-layer system covering the full lifecycle from raw intent to a deployed and verified landing zone:

- **Layer 1 — Configurator** (Python CLI with 32 subcommands + a Streamlit workbench UI). Transforms three intake paths (Azure subscription snapshots, CAF/ALZ artifacts, and text design

sheets) into a validated planning contract ("intent v0.2"), reconciles it against requirements, and materializes a draft inventory in an isolated sandbox. The configurator **never calls Azure write APIs**, and approval flags can only be set by a human.

- **Layer 2 — Deploy Framework** (a 12-stage Bash pipeline over a declarative JSON inventory). It uses up to 23 resource-type inventory files per environment (23 in the fullest) to define all infrastructure using portable relative references. Sequential numbered scripts deploy everything in dependency order, with profile-aware stages that skip themselves when not applicable.

The layers meet at exactly one auditable bridge: `emit-deploy-config` converts an approved draft into a deploy-ready configuration. The subscription ID is supplied at bridge invocation, written into the emitted configuration, and never committed.

1.3 Primary Quality Drivers

Quality attribute	Manifestation
Idempotency	Every deploy script checks before creating; all 12 stages are safe to re-run at any time
Planning safety	The planning layer performs no Azure writes; approval flags are human-only; drafts live in an isolated sandbox
Portability	Relative references replace absolute resource IDs; whole environments clone across subscriptions
Profile flexibility	One codebase serves two governed architecture profiles via config-driven branching
Auditability	Append-only JSONL audit journal in the planning layer; timestamped deployment logs; PASS/WARN/FAIL preflight reports
Operational simplicity	No daemon, no state file, no lock contention; any stage runnable independently

1.4 Outcome

The framework replaced manual portal deployment with a single command per environment. **~1-3 h depending on estate complexity; dominated by Azure control-plane provisioning.** Full estate deployment wall-clock ranges from roughly 60 minutes for a small single-site estate to about 3 hours for complex hub-and-spoke estates. The dominant cost is Azure control-plane provisioning, not framework overhead: Application Gateway and VPN Gateway each take ~40 minutes, Azure Firewall is comparable, and resource-provider registration adds 30-40 minutes when many providers are needed. Incremental runs are correspondingly shorter. Figures are qualified estimates from production deployments; no formal timing sample was collected. Environments are cloneable across subscriptions, and the complete pipeline — collect inventory from a source estate, emit a deploy configuration, deploy into an empty target subscription, verify with a preflight check — has been proven end-to-end on live Azure, with the source environment confirmed unchanged by a before-and-after resource-state comparison.

2. Purpose and Scope

2.1 Lifecycle Phases

Phase	Input	Output	Entry point
0 — Plan	Snapshot / ALZ artifact / text sheet	Draft intent v0.2	configurator CLI or UI
0a — Reconcile	Intent + optional CAF/ALZ request	Reconciliation report (zero-conflict gate)	reconcile-alz-request / reconcile-intents
0b — Materialize	Reconciled intent	Draft inventory files (sandbox)	materialize-* subcommands
0c — Bridge	Approved draft	Deploy-ready config.json + inventory	emit-deploy-config
1 — Collect	Azure subscription (live)	Inventory JSON files	collect_inventory.sh
2 — Validate	Inventory files	PASS/FAIL schema report	validate_inventory.sh
3 — Deploy	Inventory + config	Deployed infrastructure	deploy_all.sh (stages 01-12)
4 — Verify	Deployed infrastructure	PASS/WARN/FAIL preflight report	12_preflight_check.sh
5 — Manage	Environments directory	Switch, clone, migrate environments	environment scripts

2.2 Two Architecture Profiles, One Codebase

- **The Centralized Inspection profile.** Azure Firewall inspects all traffic; DNS via firewall proxy; centralized NAT; every spoke routes its default route to the firewall; spoke-to-spoke peering prohibited.
- **The Distributed Native profile.** No firewall; NSGs are the primary control; DNS Private Resolver in the hub; per-spoke NAT gateways; spoke-to-spoke peering allowed.

A single `architectureProfile` field selects the profile. Profile-specific stages detect the mismatch and exit cleanly, so the same numbered pipeline serves both.

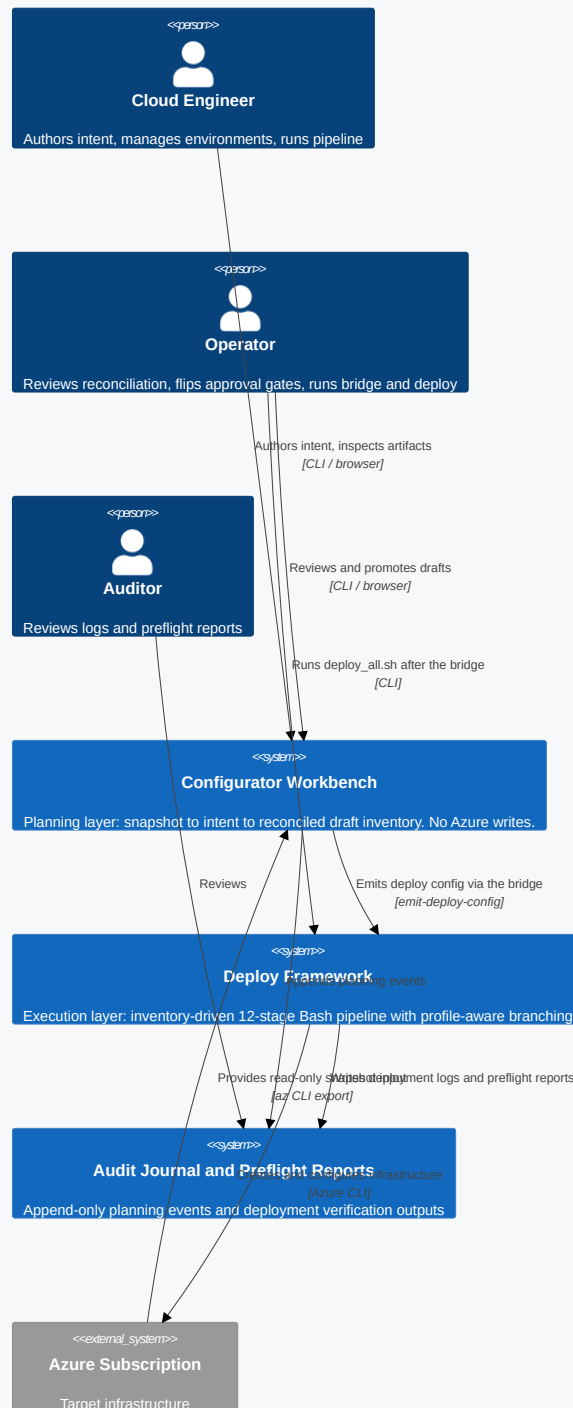
2.3 Explicitly Out of Scope

- The configurator never calls Azure write APIs. No resource creation, no ARM PUT, no role assignments from the planning layer.
- No CLI subcommand can set an approval flag to true.
- Raw customer snapshots are git-ignored and never committed; committed fixtures carry redacted identifiers and an all-zeros placeholder subscription ID.

3. System Architecture (C4)

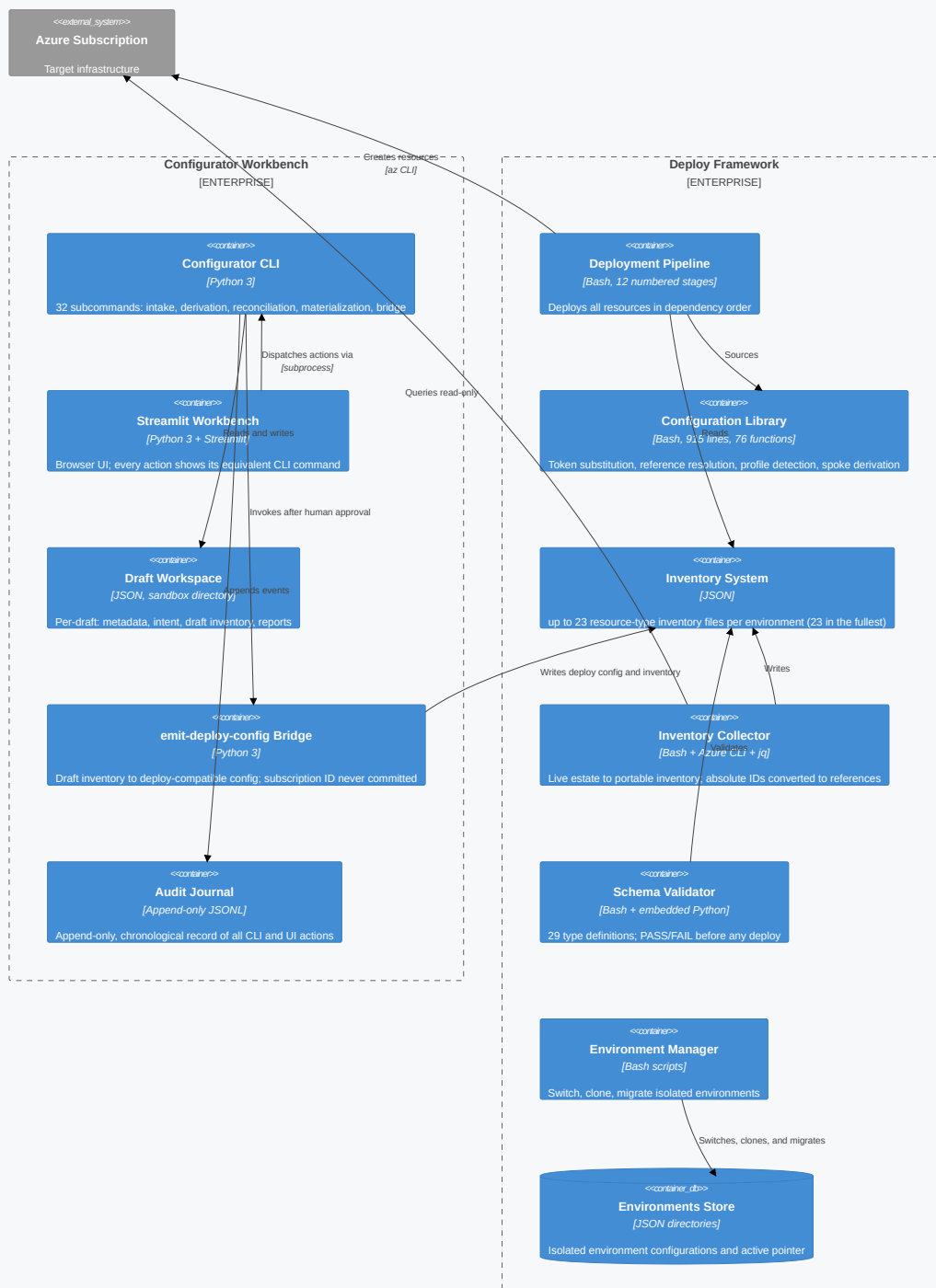
3.1 Level 1: System Context

System Context: UDF2 (Two-Layer System)

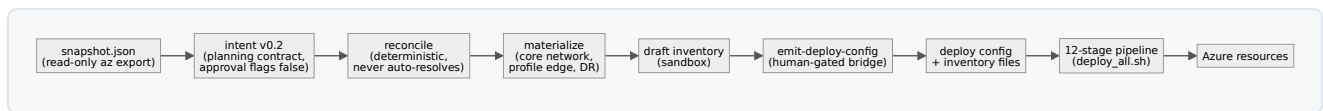


3.2 Level 2: Containers

Container Diagram: Configurator + Deploy Framework



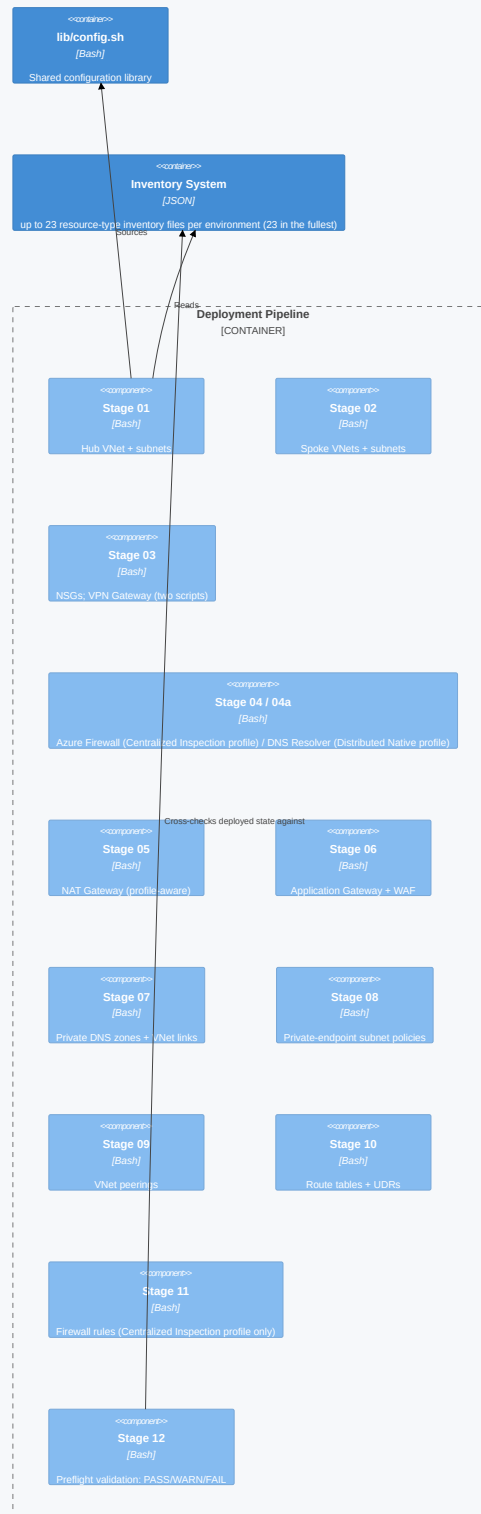
3.3 Whole-System Data Flow



The direction of trust matters: everything left of the bridge is planning and can be discarded; everything right of the bridge is execution and is guarded by the safety protocol in Section 7.

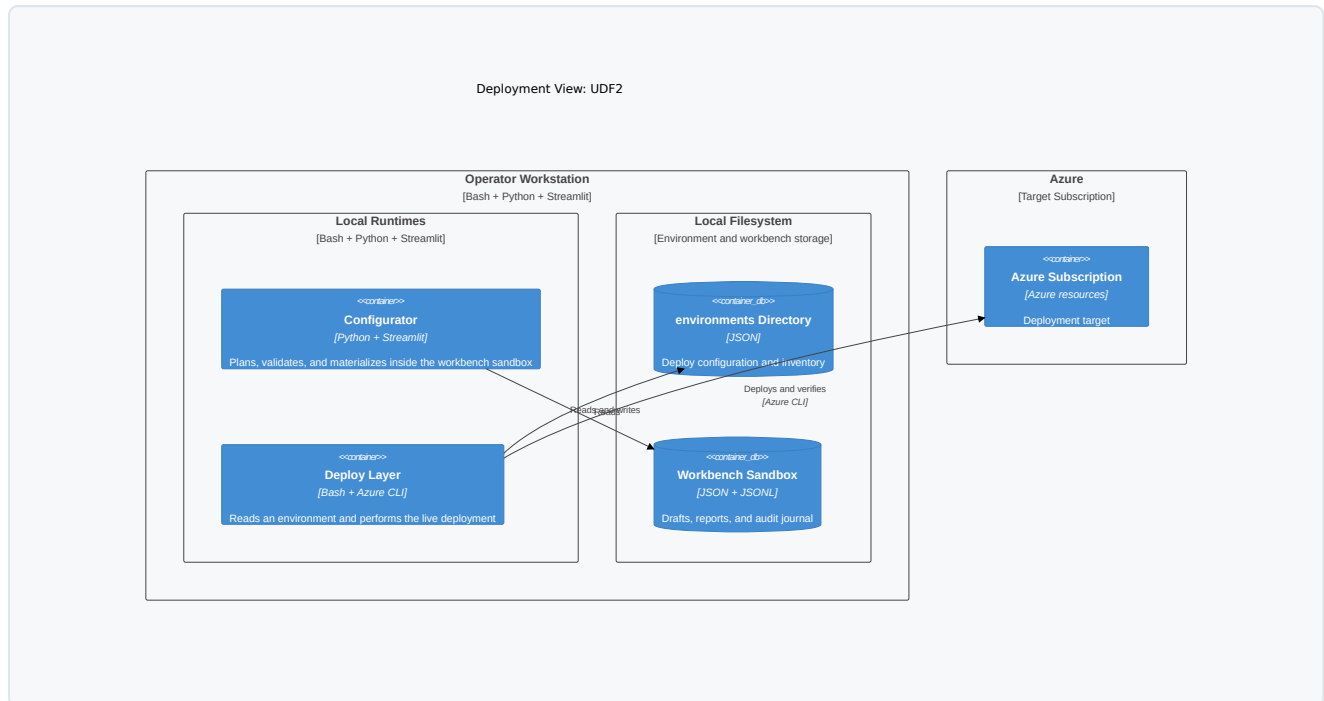
3.4 Deployment Pipeline Components

Component Diagram: 12-Stage Deployment Pipeline



Twelve numbered stages are implemented by fourteen scripts: stage 03 has separate scripts for NSGs and the VPN Gateway, and stage 04 has a profile-specific alternative (Firewall for the Centralized Inspection profile, DNS Private Resolver for the Distributed Native profile).

3.5 Deployment View

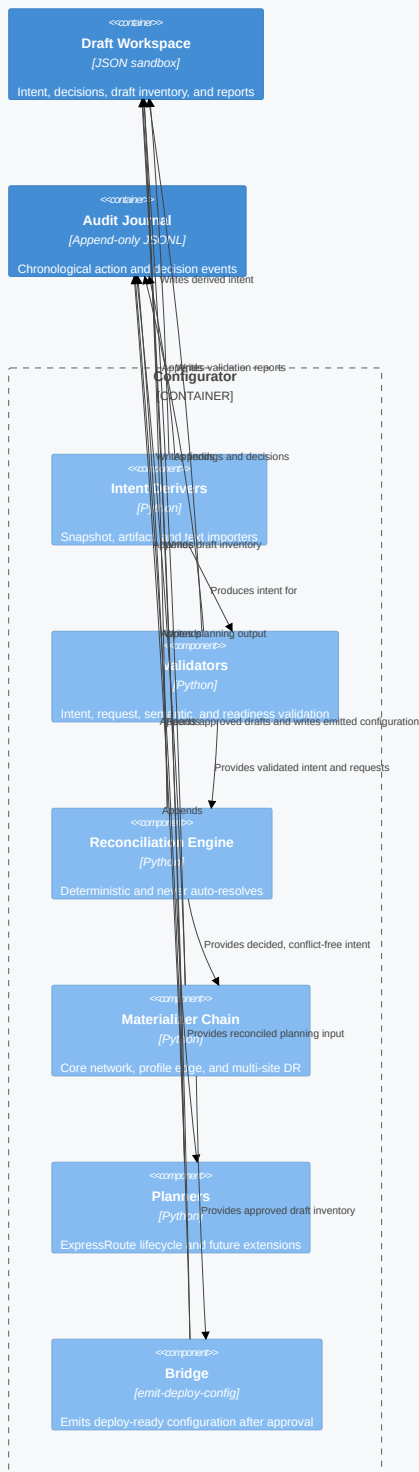


The Azure subscription is reached only by the deploy layer through the Azure CLI.

4. Layer 1: The Configurator

4.1 Configurator Component View

Component Diagram: Configurator



4.2 Three Intake Paths, One Contract

The configurator accepts intent in whatever form it exists:

1. **Azure snapshot:** a read-only export of a live estate is classified into desired-state candidates versus runtime evidence, and a structured intent is derived from it.
2. **CAF/ALZ artifact:** Bicep/ARM exports, IaC accelerator output, or blueprint artifacts are inspected, classified with a confidence score, and converted into a request manifest that an operator completes.
3. **Text design sheet:** a plain-text design document is parsed into intent.

All three paths converge on **intent v0.2**, a schema-validated, planning-only contract. Its schema states the boundary directly: it is not deployable inventory, and deploy-layer code does not read it.

A synthesized fragment:

```
{
  "schemaVersion": "0.2",
  "source": { "type": "snapshot-derived", "deriver": "snapshot_intent_deriver" },
  "project": { "name": "env-alpha", "region": "region-a" },
  "architecture": {
    "profile": "centralized",
    "materializerClass": "centralized-single-site",
    "confidence": 0.92,
    "evidence": ["AzureFirewallSubnet present in hub", "default route to firewall"]
  },
  "network": { "vnets": ["..."], "routeTables": ["..."], "peerings": ["..."] },
  "runtimeEvidence": [
    { "kind": "vm", "name": "worker-01", "desiredState": false }
  ],
  "approval": {
    "approvedForInventoryMaterialization": false,
    "approvedForAzureDeployment": false
  },
  "unresolved": []
}
```

Two details of this contract enforce the safety boundary:

- **Runtime evidence is never desired state.** Observed VMs, disks, and NICs are recorded as read-only context. A validator rejects any request that tries to promote observed state into desired state.
- **Approval flags are always false when written by software.** See Section 7.

4.3 The Reconciliation Gate

Before any materializer may run, reconciliation must report **zero error-severity findings**. The reconciler diffs a CAF/ALZ request against the derived intent (or two intents against each other) deterministically, and it **never auto-resolves**: every conflict is surfaced as a finding that requires an explicit operator decision, recorded with the decision-maker and rationale. The reconciliation report stays in the draft workspace as an audit artifact.

This is enforced as a hard gate in shared materializer code: a materializer invoked without a passing reconciliation report refuses to run.

4.4 The Materializer Chain

Materializers convert reconciled intent into draft inventory files, in dependency order: core network (resource groups, VNets, route tables), then the profile-specific edge (firewall stack or Distributed Native profile stack), then optional multi-site DR. Two planners, `plan-expressroute-lifecycle` and `plan-future-extensions`, produce planning output only and deliberately refuse full-environment materialization while unplanned resource categories remain. This is a scope-lock guard against premature "deploy everything" runs.

Every materializer run is fingerprinted: the SHA-256 of the input intent and of the reconciliation decisions is recorded with each run, so the apply step can verify that what was materialized is exactly what was reviewed. Stale drafts are detected, not trusted.

4.5 The Draft Workspace

All configurator output lives in a sandboxed workspace directory, git-ignored, with one subdirectory per draft: metadata, intent, draft inventory, reconciliation report and decisions, run logs, and an append-only audit journal. The deploy framework's environment directories are **never touched** by the configurator; a human moves approved output across that boundary, and only through the bridge.

The workbench UI adds gate-aware navigation on top: each page renders only when its prerequisites hold (intent valid, reconciliation clean, materialization fresh), and otherwise shows an explicit blocker with a redirect. The gate blocker is not an error state; it is the safety boundary made visible.

4.6 CLI Parity

Every UI action displays the equivalent CLI command. This is a deliberate governance feature: nothing the UI can do is unavailable or unreproducible from the terminal, so reviews and audits can replay any action.

5. Layer 2: The Deploy Framework

5.1 Declarative Inventory with Relative References

All infrastructure is defined in up to 23 resource-type inventory files per environment (23 in the fullest), validated against a schema with 29 type definitions. The critical design choice is **relative references**: resources point to each other by logical identity, not by absolute Azure resource ID.

```
{
  "routeTable": { "resourceGroup": "alpha-hub-rg", "name": "alpha-gateway-rt" },
  "subnet": {
    "resourceGroup": "alpha-hub-rg",
    "vnetName": "alpha-hub-vnet",
    "name": "GatewaySubnet"
  }
}
```

References are resolved to full resource IDs at deployment time, against whatever subscription is active. The consequence: an entire environment is portable. Collect in one subscription, deploy into another, and nothing in the inventory changes. The collector performs the inverse conversion: absolute IDs found in live Azure are rewritten to relative references at collection time.

5.2 Inventory Data Model

A synthesized virtual-networks fragment shows subnet-level relative references:

```
{
  "virtualNetworks": [
    {
      "resourceGroup": "alpha-hub-rg",
      "name": "alpha-hub-vnet",
      "subnets": [
        {
          "name": "GatewaySubnet",
          "networkSecurityGroup": { "resourceGroup": "alpha-hub-rg", "name": "alpha-gateway-
nsg" },
          "routeTable": { "resourceGroup": "alpha-hub-rg", "name": "alpha-gateway-rt" }
        },
        {
          "name": "workload-subnet",
          "networkSecurityGroup": { "resourceGroup": "alpha-hub-rg", "name": "alpha-workload-
nsg" },
          "routeTable": { "resourceGroup": "alpha-hub-rg", "name": "alpha-workload-rt" }
        }
      ]
    }
  ]
}
```

A synthesized route-tables fragment carries two routes and their next-hop values:

```
{
  "routeTables": [
    {
      "resourceGroup": "alpha-hub-rg",
      "name": "alpha-gateway-rt",
      "routes": [
        { "name": "default-route", "addressPrefix": "{default-cidr}", "nextHopType":
"VirtualAppliance" },
        { "name": "local-route", "addressPrefix": "{private-cidr}", "nextHopType": "VnetLocal"
      }
    }
  ]
}
```

Files cross-reference each other by logical identity, never by absolute ID. A schema with 29 type definitions validates every file before any deployment.

5.3 The Configuration Library

A shared Bash library (915 lines, 76 functions) is sourced by every stage. It owns environment resolution, architecture-profile detection, naming pattern rendering (`{prefix}`, `{env}`, `{index}` token substitution), typed config accessors, relative-reference resolution, and spoke derivation (regex matching of spoke VNets plus AKS cross-referencing, implemented in embedded Python where Bash and jq run out of expressiveness).

5.4 Token Substitution in Firewall Rules

Firewall rules in inventory reference logical entities, not addresses: tokens such as `vpn-clients`, `spokes-all`, `spoke:test`, or `hub-subnet:AzureFirewallSubnet` are resolved to concrete CIDRs at deployment time. Rules therefore survive environment cloning unchanged. The same rule set deploys into any subscription and resolves to that environment's actual address plan.

5.5 Profile Branching

Each stage begins with a profile check. In the Distributed Native profile, the firewall stage and firewall-rules stage exit cleanly; the DNS Resolver stage runs instead; NAT becomes per-spoke. The preflight check (stage 12) validates the *inverse* too: a firewall present under the Distributed Native profile is reported as a failure, not silently accepted.

5.6 Idempotency

Every stage follows check-before-create:

```
if ! az network vnet show -g "$RG" -n "$VNET" >/dev/null 2>&1; then
  # create
else
  # update or skip
fi
```

All stages are safe to re-run at any time; a re-run over unchanged infrastructure is a no-op. Failure recovery is therefore direct: fix the cause, re-run the failed stage, continue.

5.7 Deployment Timing

Qualified estimates from production deployments; no formal timing sample was collected:

Deployment shape or provisioning driver	Qualified estimate	Main reason
Small single-site estate	roughly 60 min wall-clock	Fewer control-plane resources
Complex estate (hub-and-spoke, gateway + firewall)	about 3 h wall-clock	Multiple long-running control-plane resources
Application Gateway	~40 min	Azure control-plane provisioning
VPN Gateway	~40 min	Azure control-plane provisioning
Azure Firewall	comparable to a gateway	Azure control-plane provisioning
Resource-provider registration	30–40 min	Registration when many providers are needed

~1–3 h depending on estate complexity; dominated by Azure control-plane provisioning. Full estate deployment wall-clock ranges from roughly 60 minutes for a small single-site estate to about 3 hours for complex hub-and-spoke estates. The dominant cost is Azure control-plane provisioning, not framework overhead: Application Gateway and VPN Gateway each take ~40 minutes, Azure Firewall is comparable, and resource-provider registration adds 30–40 minutes when many providers are needed. Incremental runs are correspondingly shorter. Figures are qualified estimates from production deployments; no formal timing sample was collected.

6. The Bridge: emit-deploy-config

The single crossing point between planning and execution. It reads the approved draft's inventory and emits a deploy-compatible configuration in exactly the shape the configuration library consumes. The emitted configuration on disk does carry the target subscription ID. The invariant is never-in-git, not never-on-disk, and emitted configs and environment directories are never committed.

Its behavioral contracts:

- **Hub names are inventory literals.** No token re-injection; what was reviewed is what deploys.
- **Spokes are an explicit array** of `{env, vnetName, resourceGroup}` objects. Pattern matching survives only as a legacy fallback.
- **The default location is derived from inventory resource locations**, not from a config default — the data wins over the setting.
- **The subscription ID enters at bridge invocation.** It is supplied as a flag and written to the emitted configuration, but never stored in any committed file. Committed fixtures carry an all-zeros placeholder. The emitter refuses to run without the flag.
- **A gap report is written alongside the output**, listing any profile-required inventory files still missing from the draft, so the operator sees incompleteness before copying anything into an environment.
- The emitter makes **no Azure calls** and never invokes the deploy pipeline.

7. Governance and Safety Model

This section describes the framework's safety architecture. The design assumption throughout: **the dangerous step is not planning, it is writing to a live subscription** — so every control concentrates on that boundary.

7.1 Safety Invariants

#	Invariant	Enforcement
1	No Azure writes from the planning layer	Backed by tests: the bridge test suite raises an assertion error if any test path attempts to execute an <code>az</code> command, and the UI apply tests assert that no <code>az</code> invocation appears
2	Approval flags always false from software	Unconditional; no CLI subcommand or UI button can set them
3	Raw customer snapshots never committed	Git-ignore policy; only redacted fixtures are committed
4	Draft writes confined to the sandbox	Path contract owned by one module; environments are read-only to the configurator
5	Subscription and tenant identifiers redacted	Redaction module; all-zeros placeholder policy in fixtures
6	Reconciliation gate is mandatory	Hard gate shared by all materializers; refuses to run on unresolved conflicts
7	Platform-managed resources excluded from inventory	Collection-time filter drops resources managed by other systems (for example AKS-managed groups)

7.2 Human-Only Approval

Intent carries two approval flags — one for materializing inventory, one for Azure deployment. Software writes them as false, always. Advancing either flag is a manual human edit of the draft metadata, made after review. Approval sequencing is validated too: deployment approval cannot precede inventory approval. The UI displays the flags read-only and refuses to offer the bridge until both are true.

This means the pipeline **cannot be scripted into auto-approval**. There is no flag, no environment variable, no CI setting that removes the human from the loop.

7.3 The Real-Azure Deploy Safety Protocol (Blind Triple-Check)

In the author's practice, the acting, control, and audit roles are performed by LLM agents operating under a human operator's supervision. The operator owns both approval flags and the final go decision. The protocol is equally executable by three human engineers, because the property that matters is independence of derivation, not who derives.

Development-time review gates govern authoring; they say nothing about the moment an operator points a deploy at a live subscription. A live Azure deployment changes state, can be destructive, and is billable. Deploy scripts switch the active subscription to whatever the configuration names, and in many estates the same operator identity can reach production. A single mis-pointed write can hit

production. This is the *subscription trap*, and a self-review by the acting agent is not sufficient assurance against it.

Why not simply scope the credentials?

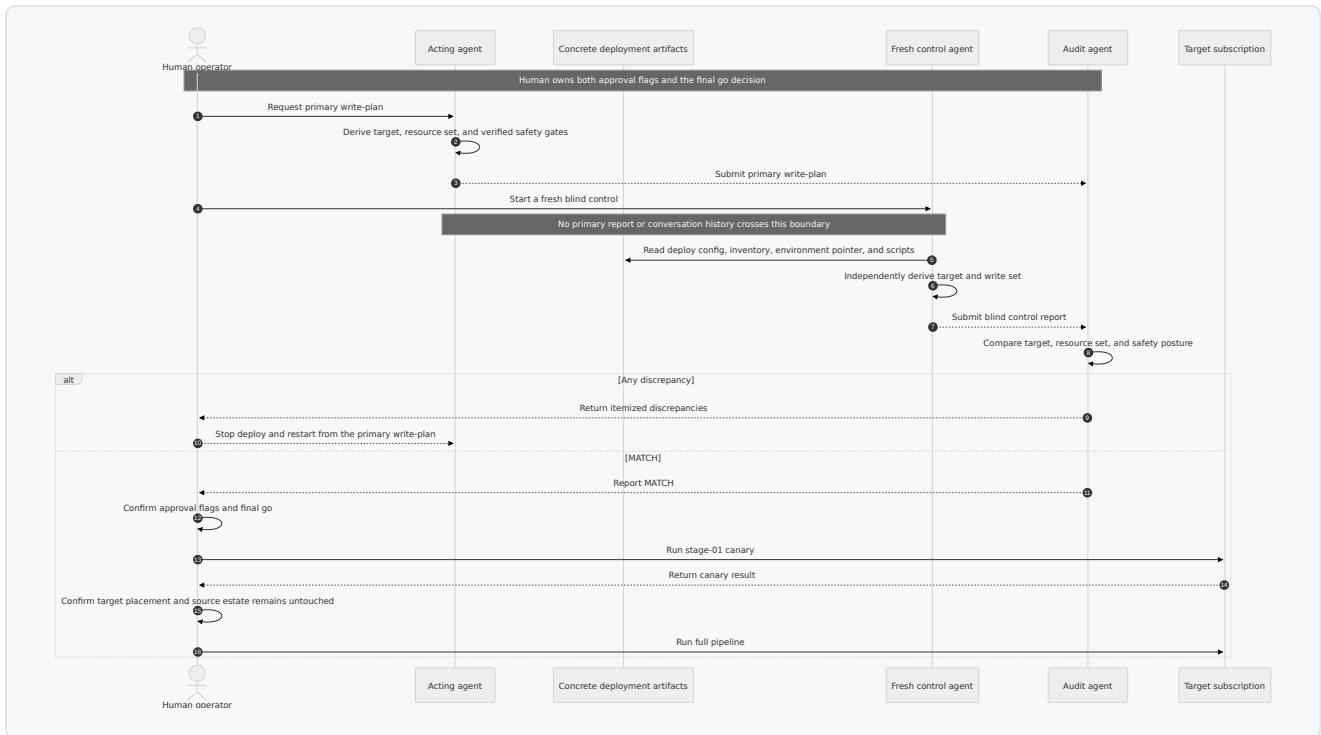
Scoped service principals, PIM-style just-in-time elevation, and policy deny assignments are the primary control wherever the estate provides them. This protocol is defense-in-depth for the consulting reality in which one operator identity often spans estates.

The framework therefore mandates a **blind triple-check** before any live write:

1. **Primary write-plan.** The acting agent produces a written report: the target subscription (stated explicitly, by ID and name), the active environment, every resource group and resource that will be created or modified, and the safety gates verified.
2. **Blind control.** A second, fresh agent — blind to the primary's report and to the conversation history — receives only the concrete artifacts: the deploy configuration, the inventory directory, the environment pointer, and the scripts to be run. Its instruction: independently derive exactly what these artifacts will write, and to which subscription. It must not assume; it must derive.
3. **Audit and gate.** A third agent compares the two reports: same target subscription, same resource set, same safety posture. The outcome is MATCH or an itemized list of discrepancies. **Any divergence stops the deploy** and returns the work to the primary; the protocol re-runs from step 1.
4. **Canary, then deploy.** Even after a MATCH, the first write is a stage-01 canary (hub resource group and VNet only), confirmed to have landed in the target subscription with the source estate untouched, before the full pipeline runs.

Runtime view: blind triple-check before a live write

The control agent is a fresh invocation. It receives neither the primary report nor the conversation history. It derives the target and write set from the concrete deployment artifacts, and the audit agent compares the two independently produced reports before the human operator makes the final go decision.



Two structural guards back the protocol:

- **Portability enforcement.** The inventory that is about to deploy must contain no concrete subscription in any resource path. The collector rewrites absolute subscription paths to a placeholder at collection time, and the validator *fails* if any concrete subscription path remains. Sub-portability is enforced by construction rather than assumed.
- **Cost hygiene.** Gateways, firewalls, and application gateways bill hourly; the protocol pairs every live test with prompt teardown — which is itself subject to the same subscription verification, because a mis-pointed cleanup is the same trap in reverse.

The value of blindness is worth stating: the control agent cannot inherit the primary's assumptions, because it never sees them. Agreement between two independently derived write-plans is evidence; agreement between an author and its own summary is not.

7.4 Engineering-Process Governance

The framework itself was delivered under a gate-based process: each stage was authored as a written specification, independently scope-checked, implemented, and then reviewed by a reviewer with no authoring context, with per-stage records retained in the repository. The same author-never-reviews principle that governs live deploys (Section 7.3) governed the code that performs them.

7.5 Auditability

Every configurator action — CLI or UI — appends a timestamped event to an append-only JSONL journal. Decision records carry actor and rationale. Materializer runs carry input fingerprints. Deployment runs write timestamped logs, and the preflight stage produces an explicit PASS/WARN/FAIL report per invariant. The result is a continuous evidence chain from "what was asked for" to "what is deployed".

8. Multi-Environment Management

Environments are isolated directories, each holding its own configuration and inventory. An environment pointer file selects the active one; switching is atomic; symlinks preserve backward compatibility with single-environment layouts. A one-time migration script converts a legacy single-environment layout into the `environments` directory structure.

```
./list_environments.sh                # show all, mark active
./switch_environment.sh env-alpha     # activate
./init_environment.sh env-clone --from env-alpha \
  --subscription SUB_ID --prefix clone # clone across subscriptions
```

Cloning plus relative references is the workflow that makes the safety protocol practical: a production estate is collected, cloned into a test environment pointing at an empty subscription, deployed, verified, and torn down while the source environment remains read-only.

9. Testing and Verification

All figures verified by running the suites on 2026-08-13.

9.1 Configurator and UI Test Suite

664 automated tests cover the configurator core and the workbench UI: intent derivation, validators, reconciliation, materializer gates, the bridge contract, draft lifecycle, gate-blocker behavior of every UI page, and locking semantics. The verification run finished 658 passed, 5 skipped, 1 failed. The single failure is a wall-clock launch-latency smoke test that fails under full-suite load and passes in isolation.

Safety invariants are tested *negatively* as well: fixtures exist specifically to prove that deployment approval without inventory approval is rejected, that runtime evidence marked as desired state is rejected, and that materializers refuse unreconciled input.

9.2 Deploy-Layer Verification

~1-3 h depending on estate complexity; dominated by Azure control-plane provisioning. Full estate deployment wall-clock ranges from roughly 60 minutes for a small single-site estate to about 3 hours for complex hub-and-spoke estates. The dominant cost is Azure control-plane provisioning, not framework overhead: Application Gateway and VPN Gateway each take ~40 minutes, Azure Firewall is comparable, and resource-provider registration adds 30-40 minutes when many providers are needed. Incremental runs are correspondingly shorter. Figures are qualified estimates from production deployments; no formal timing sample was collected.

The deploy layer is verified by construction and by preflight:

- **Schema validation** (29 type definitions) gates every deploy; structurally invalid inventory never reaches Azure.
- **The stage-12 preflight check** validates the deployed estate against the selected architecture profile — including negative checks (a firewall present in the Distributed Native profile is a FAIL).
- **The end-to-end path was proven on live Azure:** collect from a real source estate, emit the deploy configuration, deploy into an empty test subscription behind the blind triple-check and a

stage-01 canary, verify by preflight, and confirm the source estate untouched by comparing resource state before and after.

illustrative excerpt, synthesized to match the report format.

```
PASS firewall present and firewall subnet routed
PASS DNS proxy enabled on firewall
WARN spoke without dedicated NAT gateway
Summary: PASS 2, WARN 1, FAIL 0
```

10. Architecture Decision Records

ADR-001: Sequential Bash + Azure CLI over Terraform/Bicep

Context. The pipeline needs fine-grained control of deployment order and profile-based branching; roughly half of the stages behave differently per profile.

Decision. Numbered Bash scripts calling the Azure CLI directly, orchestrated by one runner.

Consequences. Immediate error feedback, explicit profile branching, no state file and no lock contention, and any stage runnable alone. Trade-off: no automatic drift detection and no dependency graph — ordering lives in the numbering, and the preflight stage compensates by validating the deployed result.

ADR-002: Declarative JSON Inventory over ARM Template Exports

Decision. A custom inventory format, one file per resource type, schema-validated.

Consequences. Operators review and edit infrastructure as data; files are merge-friendly; validation catches structural errors before any Azure call. Trade-off: cross-resource relationships span files.

ADR-003: Relative References over Absolute Resource IDs

Context. Absolute IDs bound the inventory to one subscription.

Decision. `{resourceGroup, name}` reference objects, resolved at deploy time.

Consequences. Environment cloning across subscriptions became a supported workflow instead of a rewrite exercise; the collector converts in the opposite direction at collection time; resolution handles both formats for backward compatibility.

ADR-004: Multi-Environment via Directory Isolation

Decision. Per-environment directories plus an active-environment pointer, with symlinks for backward compatibility.

Consequences. Environment state is independent; switching is atomic; cloning never touches the source. Trade-off: symlink indirection.

ADR-005: Architecture Profiles as Config-Driven Branching

Decision. One profile field drives conditional execution inside each stage, rather than maintaining two pipelines.

Consequences. No code duplication across profiles; profile-specific stages skip themselves; the preflight check enforces profile invariants in both directions. Trade-off: per-script profile boilerplate.

ADR-006: Token Substitution over Static CIDRs in Firewall Rules

Decision. A small token language for rule sources and destinations, resolved at deploy time.

Consequences. Rules are declarative and portable across environments. Trade-off: operators learn the token vocabulary.

ADR-007: Intent v0.2 as a Planning-Only Contract

Decision. The planning contract is explicitly non-deployable, and software always writes its approval flags as false.

Consequences. A clear semantic boundary between plans and deployable artifacts; unreviewed plans cannot deploy; the CLI cannot be scripted into auto-approval. Trade-off: approval is always a manual step — which is the point.

ADR-008: Mandatory Reconciliation Before Materialization

Decision. All materializers share a hard gate requiring a conflict-free reconciliation report with every finding explicitly decided.

Consequences. Conflicts cannot silently reach inventory; the planning pipeline is strictly ordered. Trade-off: reconciliation runs even for small environments.

ADR-009: One Bridge Between the Layers

Decision. A single, explicit subcommand is the only path from draft to deploy-ready configuration; all other paths are prohibited.

Consequences. One auditable handoff; reviewed literals (hub names, explicit spoke arrays) flow through unchanged; the subscription ID enters at bridge invocation and is never committed. Trade-off: one more explicit step, which keeps the handoff deliberate.

ADR-010: Draft Sandbox Never Touches Environments

Decision. All planning writes are confined to a git-ignored workspace; the environments directory is read-only from the planning layer.

Consequences. No planning bug can corrupt live environment state; drafts are disposable; the promotion step is a deliberate human action.

ADR-011: Streamlit Workbench with CLI Parity

Context. The workbench must call Python tooling, render reports, and expose gates; an earlier static-frontend placeholder could not.

Decision. A Streamlit workbench where every action renders its CLI equivalent.

Consequences. Python-native UI, no frontend/backend boundary to maintain, and full reproducibility of any UI action from the terminal. Trade-off: a heavier UI dependency.

11. Limitations and Failure Modes

11.1 Known Limitations

1. **No drift detection.** The framework deploys and verifies; it does not continuously reconcile live state against inventory. Drift is caught only when inventory is re-collected or preflight is re-run.
2. **Ordering by convention.** Stage numbering encodes the dependency graph; a new resource type with unusual dependencies requires deliberate placement.
3. **Deployment wall-clock is Azure-bound.** VPN gateways alone take 30–45 minutes; the framework cannot compress Azure provisioning times.
4. **ExpressRoute is carrier-gated.** Circuit provisioning involves an external provider; the framework plans and documents ER but its deployment remains a manual, externally dependent procedure.
5. **Schema validator implements a subset.** The custom validator covers the JSON Schema features the inventory actually uses; exotic schema constructs are not supported.
6. **The human steps are real steps.** Approval, promotion, and the triple-check protocol add minutes of deliberate friction before any live write. This is a design choice, not an oversight; environments where that friction is unacceptable would need a different risk model.

11.2 Failure Modes

Planning layer.

Failure	Behavior	Recovery
Materializer invoked without clean reconciliation	Hard gate refuses; distinct exit code	Reconcile; decide every finding
Prior materializer stage missing	Gate reports the specific missing stage	Run stages in order
Profile contradiction in intent (for example a firewall route under the Distributed Native profile)	Materializer refuses	Correct the intent
Bridge invoked without a subscription ID	Refuses to write anything	Supply the runtime flag
Stale draft (intent changed after materialization)	Fingerprint mismatch detected	Re-materialize

Deploy layer.

Failure	Behavior	Recovery
Missing or invalid inventory file	Stage exits with a specific message before any Azure call	Collect or fix inventory
Azure auth expired	Fails at stage start	Re-authenticate, re-run
Throttling or timeout	Stage fails; completed stages remain valid	Wait, re-run the failed stage
Profile mismatch in deployed estate	Preflight reports FAIL/WARN per invariant	Remediate per report
Partial deployment	All completed resources remain; re-run skips them	Fix cause, re-run

11.3 Recovery Model

Stage failed?	Fix the cause, re-run that stage. Idempotency makes this safe; completed stages are no-ops on re-run.
Wrong plan?	Discard the draft. Nothing outside the sandbox changed.
Wrong target?	The protocol exists precisely so this is caught before the first write. If the canary lands wrong, stop and tear down one resource group, not an estate.

12. Lessons Learned

- Separate planning from execution with a physical boundary, not a convention.** The planning layer cannot write to Azure because no code path exists that could, not because policy forbids it. Invariants enforced by construction do not decay as long as a guard keeps them true; here the guard is the test suite. The bridge tests raise an assertion error on any attempted `az` command, and UI apply tests assert that no `az` invocation appears.
- Approval must be un-scriptable.** The moment an approval flag can be set by software, someone's automation will set it. Making approval a human file edit is crude, and that crudeness is load-bearing.
- Blind review is qualitatively different from more review.** The triple-check works because the control agent derives its write-plan from artifacts alone. Independent derivation catches wrong-subscription and wrong-scope errors that any number of same-context re-reads would miss.
- Portability is a safety property, not just a convenience.** Removing concrete subscription IDs from inventory made environments cloneable — and simultaneously removed the class of accidents where a leaked absolute ID silently targets the wrong subscription. Enforcing it in the validator turned a convention into a gate.
- Idempotency turns failure into a non-event.** Check-before-create everywhere means the answer to nearly every deployment failure is "fix and re-run", with no state reconstruction and no fear of duplicates.

6. **Make the gate visible, not just effective.** UI pages that explain *why* they are blocked (and where to go) turned the safety model from an obstacle into documentation. Operators stopped working around gates when the gates explained themselves.
7. **The data should outrank the config.** Deriving locations and names from reviewed inventory literals — rather than re-generating them from patterns at deploy time — eliminated a whole category of "the plan said A, the deploy did B" discrepancies.
8. **Canaries are cheap; estates are not.** One resource group deployed and verified before the other eleven stages costs minutes and bounds the impact of wrong-target mistakes and other failures visible in the first stage.

13. Roadmap: Unified Toolchain

Status: vision, not yet implemented.

The planning layer already accepts the documentation pipeline snapshot format as one of its intake paths. The unimplemented part is the shared topology library and round-trip validation. The deployment framework and its companion documentation pipeline are two halves of one lifecycle: deploy from inventory, document from snapshot. The companion document is *Azure Auto-Documentation and Diagramming Framework - Architecture Reference*. Both tools remain independently usable; the unified layer is an orchestration skin, not a rewrite.

Appendix A: CLI Reference

```
# Planning layer (configurator)
udf-configurator create-draft --env env-alpha
udf-configurator derive-intent-from-snapshot --draft draft-id --snapshot snapshot.json
udf-configurator validate-intent --draft draft-id
udf-configurator reconcile-alz-request --draft draft-id --request request.json
udf-configurator materialize-core-network --draft draft-id
udf-configurator materialize-centralized-edge --draft draft-id
# (human review; approval flags advanced manually)
udf-configurator emit-deploy-config --draft draft-id --subscription-id SUB_ID

# Deploy layer
./collect_inventory.sh           # live estate -> portable inventory
./validate_inventory.sh         # schema gate
./deploy_all.sh                 # stages 01-12 with confirmation
./l2_preflight_check.sh         # profile-aware verification
./cleanup.sh                    # full teardown (same subscription checks)

# Environment management
./list_environments.sh
./switch_environment.sh env-alpha
./init_environment.sh env-clone --from env-alpha --subscription SUB_ID
```

Appendix B: Codebase Shape

Verified counts (2026-08-13):

Element	Count	Note
Configurator CLI subcommands	32	Enumerated by the CLI help
Configurator core modules	36	Python modules behind the CLI
Deploy pipeline scripts	14	Implementing 12 numbered stages
Configuration library	915 lines, 76 functions	Shared Bash library
Inventory files	up to 23 resource-type inventory files per environment (23 in the fullest)	One JSON file per resource type
Inventory schema	1,437 lines, 29 type definitions	Custom stdlib-only validator
Automated tests (configurator + UI)	664 (658 passed, 5 skipped, 1 failed: load-sensitive smoke, passes in isolation)	Verification run

End of document. All names are placeholders; figures verified against the repository on 2026-08-13; deployment durations are documented estimates from real deployments.